Giftogram – Technical Assessment

Overview

For this assessment, you are required to develop a simple RESTful API backend for a mobile chat application.

Your solution must:

- Interact with a **MySQL database**
- Include a properly structured **database schema**
- Return **valid JSON responses** for all endpoints
- Be built using **JavaScript or PHP**
    - You may use any framework of your choice
    - You may also hand-code everything without a framework

Application Requirements

The chat application must allow users to:

- Register an account
- Log in
- Send individual messages to other users
- View message history between two users
- List all registered users (excluding the requester)

You are free to architect the backend and database structure as you see fit, as long as the required endpoints are implemented.

Error Response Format

Some API endpoints must return an error response when appropriate (e.g., failed login).

All error responses must include:

- `error_code`
- `error_title`
- `error_message`

Example Error Response

```
{
  "error_code": 101,
  "error_title": "Login Failure",
  "error_message": "Email or Password was Invalid!"
}
```

You may define your own error codes and messages.

Required API Endpoints

---

1. Register User

Endpoint: `/register`
Method: `POST`
Description: Registers a brand new user.

Example Request Body

```
{
  "email": "info@giftogram.com",
  "password": "Test123",
  "first_name": "John",
  "last_name": "Doe"
}
```

Example Response

```
{
  "user_id": 1,
  "email": "info@giftogram.com",
  "first_name": "John",
  "last_name": "Doe"
}
```

---

2. Login

Endpoint: `/login`
Method: `POST`
Description: Authenticates a user. If login fails, return a properly structured error response.

Example Request Body

```
{
  "email": "info@giftogram.com",
  "password": "Test123"
}
```

Example Successful Response

```
{
  "user_id": 1,
  "email": "info@giftogram.com",
  "first_name": "John",
  "last_name": "Doe"
}
```

---

3. View Messages

Endpoint: `/view_messages`
Method: `GET`
Description: Returns all messages exchanged between two users, ordered by date.

Example Request Parameters

```
{
  "user_id_a": "1",
  "user_id_b": "2"
}
```

Example Response

```
{
  "messages": [
    {
      "message_id": 1,
      "sender_user_id": 1,
      "message": "Hey what is up?",
      "epoch": 1429220026
    },
    {
      "message_id": 2,
      "sender_user_id": 2,
      "message": "Not much, how are you?",
      "epoch": 1429221050
    }
  ]
}
```

---

4. Send Message

Endpoint: `/send_message`
Method: `POST`
Description: Sends a message from one user to another. Returns a success response if sent successfully.

Example Request Body

```
{
  "sender_user_id": 1,
  "receiver_user_id": 2,
  "message": "Example text"
}
```

Example Success Response

```
{
  "success_code": 200,
  "success_title": "Message Sent",
  "success_message": "Message was sent successfully"
}
```

5. List All Users

Endpoint: `/list_all_users`
Method: `GET`
Description: Returns all registered users excluding the requester.

Example Request Parameters

```
{
  "requester_user_id": 3
}
```

Example Response

```
{
  "users": [
    {
      "user_id": 1,
      "email": "ppeck@giftogram.com",
      "first_name": "Preston",
      "last_name": "Peck"
    },
    {
      "user_id": 2,
      "email": "jgreen@giftogram.com",
      "first_name": "Jake",
      "last_name": "Green"
    }
  ]
}
```

Deliverables

Please submit one of the following:

- **(Preferred)** A link to a public GitHub (or similar) repository containing your code
- A ZIP file containing all project files

Commit early and commit often, we want to see you work through your issues via code. If you are using a framework, make an initial commit containing the base framework installation before making any modifications

---

Database Submission

Include a **SQL dump** of your MySQL database so we can review your schema design.

You may:

- Include the SQL dump in your repository
  (Not best practice for production, but acceptable for this demo.)

Repository Requirements

Your repository (or ZIP file) must include a `README.md` file containing:

- Your name
- How long it took you to complete the test
- A summary of the steps you took from start to finish
- Any issues you see with the endpoint structure
- Suggested improvements regarding:
  - Security
  - Usability
  - API design